

Language Subsets & Coding Standards

Håkan Forsberg
2026-02-23

Agenda

- Who Am I?
- Introduction to language subsets and coding standards
- Coding standards
 - Some examples
 - Mostly focused on HDL languages

Håkan Forsberg

- MDU

- Associate Professor (40% research, 60% education)
- Program responsible for the Master of Science in Engineering program in Dependable Systems
- Subject representative, Aeronautical Engineering

- Industrial Experience

- +2 years at Spectronic (former Swedish cell phone developer)
- 18+5 years at Saab (1996 – 2014) (2019-2023 & 2026 as consultant)
 - - Design Engineer, system design and VHDL programmer for “complex” PLDs
 - - Principal Engineer, HW
 - - Technical Specialist in Safety-Critical Computers and Electronics
 - - Special consultant work – future systems / products



What do You know about ...

- MISRA C?

*MISRA C is a software development standard for the C programming language. Its aims are to facilitate code **portability** and **reliability** in the context of embedded systems, specifically those systems programmed in ANSI C*

Vittorio Giovara, Politecnico di Torino

- Ravenscar?

*The purpose of the Ravenscar profile is to restrict the use of many tasking facilities so that the effect of the program is **predictable***

John Barnes

*The Ravenscar profile is a subset of the Ada - tasking features designed for **safety critical hard real-time computing***

Part of the Ada 2012 standard nowadays - it uses compiler restrictions for the Ada language

Source: Wikipedia

What do You know about ...

- IEC61508?

IEC61508 is the general industry standard for system safety –
It includes guidance for SW programming and HW
programming (HDL)

- Linting tools?

*A lint tool is a tool that analyzes source code to flag
programming errors, bugs, stylistic errors, and suspicious
constructs*

Source: Wikipedia

Language Subsets vs Coding Standards



- A subset of a language, is a “new language” that never allows you to use other than the subset of the original language in the code
- A coding standard constraints the programmer to use safe coding styles and enforces the use of clearly readable code for a specific language that could be dangerous to use otherwise
 - It works perfectly well to use coding standards on subsets of languages
 - So, if no one (human or tool) checks that the programmer has followed the coding standard, there is a potential risk that the code contains bad software/hardware design which potentially leads to design errors/bugs

Why do You need to know ...

about language subsets and programming standards?

- Because as soon as you are going to do some programming for the industry you will use some kind of language subsets or coding standards
 - explanation follows

Why do You need to know ...

about language subsets and programming standards?

Example 1 – C code

```
for (si=0; (si<5); si++){
```

```
...
```

```
    si=si+1;  
}
```

“Identifiers modified within the increment expression of a loop header shall not be modified inside the block controlled by that loop header”.

Commenting Code is also Part of Good Coding Practices - I

- The basic tenets of commenting code:
 - Make them brief
 - Keep them relevant
 - Use them liberally, but not to excess

Source: B.J. Keeton, “How to comment your code like a pro: Best practices and good habits” April 3, 2019

- **Never add stupid comments!**

```
/*This code sucks, you know it and I know it. Move on and call me  
an idiot later.*/
```

Source: Bill Sourour, “Putting comments in code: the good, the bad, and the ugly,” April 20, 2017

Come on ...

“Good code is self-documenting!”

It is cliché!

Self-Explainable?

- How about this 160-byte of C code? Self-explainable? Would you like to find this kind of code in a safety-critical project?

```
int a=10000,b,c=2800,d,e,f[2801],g;main(){for(;b-c;)f[b++]=a/5;  
for(;d=0,g=c*2;c-=14,printf("%.4d",e+d/a),e=d%a)for(b=c;d+=f[b]*a,  
f[b]=d%--g,d/=g--,--b;d*=b);}
```

Have you figured out what it does?

The code is written by Dik T. Winter and it computes/prints the 800 first digits of pi!

Commenting Code is also Part of Good Coding Practices - II

Be careful when the coding standards require a certain amount (in percentage) of comments

It is not the quantity but rather the quality of the comments that counts

How about ChatGPT's Talented Code Commenting Feature? ChatGPT 3

- Simple Example, Python code first generated without comment and then with comment

"Write a python code that calculates pi"

```
def calculate_pi(n):
    pi = 0
    for i in range(n):
        pi += (-1)**i / (2 * i + 1)
    pi *= 4
    return pi

print(calculate_pi(10000))
```

```
def calculate_pi(n):
    # initialize the value of pi
    pi = 0

    # iterate n times
    for i in range(n):
        # add the next term of the Leibniz formula
        pi += (-1)**i / (2 * i + 1)

    # multiply by 4 to get the final value of pi
    pi *= 4

    # return the calculated value of pi
    return pi

# test the code with n=10000
print(calculate_pi(10000))
```

"add comments to the code"

Matlab Example – ChatGPT 3

Matlab code generated
without asking for
comments

```
% Define the input signal
t = 0:0.01:1;
x = sin(2*pi*t);

% Calculate the Fourier transform
X = fft(x);

% Plot the result
figure;
plot(abs(X));
title('Fourier Transform of the Signal');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
```

... and with comments

```
% Define the input signal
t = 0:0.01:1; % Create a time vector ranging from 0 to 1 with a step
size of 0.01
x = sin(2*pi*t); % Generate a sinusoidal signal with frequency of 1
Hz

% Calculate the Fourier transform
X = fft(x); % Apply the Fast Fourier Transform (FFT) algorithm to
the signal

% Plot the result
figure; % Create a new figure
plot(abs(X)); % Plot the magnitude of the Fourier transform
title('Fourier Transform of the Signal'); % Add a title to the plot
xlabel('Frequency (Hz)'); % Label the x-axis
ylabel('Magnitude'); % Label the y-axis
```

Use of AI Tools for Safety-Critical Applications

- While simple examples may work - **never use/trust AI generated code or comments** for safety-critical projects! (At least not yet!)
 - Humans are still better in coding special conditions
- How about using AI to validate code or to check it follows some rules?
 - Yes, but ordinary code checking tools can be completely assessed, thus better to use
 - Also, be careful – humans trust more in AI than in other humans
- How about tracing requirements to code (part of validation) using AI?
 - For safety-critical projects, code should be traceable without the help of AI!
 - (Might be useful in certain situations)

Constraining Software Code

for critical systems

- Why do we want to constrain or subset a language?
 - Easy to make typing mistakes in C that lead to perfectly valid code
 - C easily misunderstood by programmers
 - Deterministic behavior
 - Analyzability
 - Readability
 - Portability
 - ...
- Are there any differences in software languages in terms of making programming mistakes?
 - Yes!
 - Ada versus C
 - VHDL versus Verilog

Constraining Software Code for critical systems - C

- Examples of subset software languages / coding standards
 - MISRA C (coding standard)
 - Made C useful for safety-critical systems
 - Originally for the automotive domain
 - Latest version MISRA C, 2025
 - Nowadays used in several domains (aerospace, medical devices, financial, industrial control systems, ...)
 - MISRA C++ (coding standard)
 - Latest version, October 2023

MISRA C++:2023 Guidelines for the use of C++:17 in critical systems

Published in October 2023, this is the latest and current edition of MISRA C++. It is specifically targeting the 2017 language version (C++:17) as defined by ISO/IEC 14882:2017.

Constraining Software Code

for critical systems - Ada

- Examples of subset software languages / coding standards
 - SPARK (Subset of Ada)
 - Latest version SPARK 2014, used for high integrity software, for math approved evidence, it is predictable and highly reliable
 - Ravenscar (Subset of Ada)
 - Used for safety-critical hard real-time computing
 - ISO/IEC TS 24718:2025 - Guidance for the use of the Ada Ravenscar Profile in high integrity systems

How about HDL Coding Standards?

- The same reasons as for software + **for avoiding dangerous hardware constructs**
- - Come on, the synthesis and place & route tools capture all these dangerous hardware constructs, for sure!
 - True or false?
- The avionics industry may be the most stringent industry for both software and hardware coding standards
 - for other domains it is more common to use coding standards and code reviews for software only
 - and if they use it for hardware, it typically not include rules for combatting design errors, only dangerous hardware constructs or for portability issues

Who Ensures the Code Follows a Certain Standard?

- One aircraft manufacturer developed a safety-critical hardware system for another aircraft manufacturer (in year 2008)
 - DAL C – “medium” safety-critical system
 - Only code for one FPGA
 - Each time the VHDL code was formally reviewed
 - 3999 manual checks (human) toward The HDL standard
 - #Rules x VHDL files
 - (+ 558 code validation checks)

Do you foresee any problem doing this!!!!



Linting Tools

- To avoid using manual reviews for most of the code, a lint tool is used
- A lint tool analyzes the source code to find
 - programming errors,
 - bugs,
 - stylistic errors, and
 - suspicious constructs
- Lint tools for HDL may also include dangerous hardware constructs

Linting Tools for HW

- Many HDL tool vendors support adaptive linting tools
 - Questa Autocheck - Siemens EDA ...
 - If account is created further information can be downloaded
<https://eda.sw.siemens.com/en-US/ic/questa/formal-verification/autocheck/>
 - <https://insights.sigasi.com/manual/linting-vhdl/#list-of-vhdl-code-rules>
Sigasi Studio may be offered for free for universities

HW Lint Tools in Avionics

- HW tools divided into two groups
 - Design tools
 - Verification tools (include tools used for validation purposes)
- All HW tools need to be assessed
 - I.e., we cannot rely on the tools
 - Different strategies to ensure no bugs in tools
- A HW lint tool may be used if evidence is shown that the lint tool works correctly
 - A HW lint tool may be used for both design and verification code (testbench code)
 - A HW lint tool is considered a verification tool since it “verifies” the design as soon as the RTL code is written

Exercise – The Lint tool should have found at least three violations in this VHDL code – can you find them?

```
IF (reset = '1') THEN
-- Default Reset Values
beep <= '0';
signal_1 <= '0';
signal_2 <= '1';
beep <= '1';
elsif (falling_edge(clk)) then
```

What About Python?

- Coding standards and best practices exist for Python as well
- See: <https://www.zenesys.com/blog/coding-standards-and-best-practices-for-python-code-quality>

```
*****Inspiron-N4010:~$ python3
Python 3.8.5 (default, May 27 2021, 13:30:53)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>> █
```

E.g., use the ‘with’ statement while opening a file, the ‘with’ statement closes the file even if there is an exception raised

Zen of Python by
Tim Peters, 2004

Example - Coding Standards for Safety-Critical Designs

- IEC 61508 – General industry standard for system safety
 - Includes detailed guidance on SW design and HDL design, e.g.
 - Use of structured design,
 - Restricted use of asynchronous constructs,
 - Design for testability,
 - Coding guidelines
 - Use of tools to check coding guidelines

IEC 61508 Guidance Examples for Software

Description: The following principles indicate how safety-related coding rules (for any programming language) can assist in complying with the IEC 61508-3 normative requirements and in achieving the informative “desirable properties” (see Annex F). Account should be taken of available support tools.

<i>IEC 61508-3 Requirements & Recommendations</i>	<i>Coding Standards Suggestions</i>
Modular approach (Table A.2-7, Table A.4-4)	Software module size limit (Table B.9–1) and software complexity control (Table B.9–2). Examples: • Specification of “local” size and complexity metrics and limits (for modules) • Specification of “global” complexity metrics and limits (for overall modules organisation) • Parameter number limit / fixed number of subprogram parameters (Table B.9–4) Information hiding/encapsulation (Table B.9–3): e.g., incentives for using particular language features. Fully defined interface (Table B.9–6). Examples: • Explicit specification of function signatures • Failure assertion programming (Table A.2-3a) and data verification (7.9.2.7), with explicit specification of pre-conditions and post-conditions for functions, of assertions, of data types invariants
Code understandability • Promote code understandability (7.4.4.13) • Readable, understandable and testable (7.4.6)	Naming conventions promoting meaningful, unambiguous names. Example: avoidance of names that could be confounded (e.g., IO and I0). Symbolic names for numeric values. Procedures and guidelines for source code documentation (7.4.4.13). For example: • Explain why’s and meanings (and not only what), • Caveats • Side effects Where practicable, the following information shall be contained in the source code (7.4.4.13): • Legal entity (for example: company, author(s), etc.) • Description • Inputs and outputs • Configuration management history (See also Modular Approach)

IEC 61508 Guidance Examples for Software cont.

Good programming practice (7.4.4.13)	When applicable: • Coding guidelines to ensure that, when necessary, floating point expressions are evaluated in the right order (e.g., “a-b+c” is not always equal to “a+c-b”) • In floating point comparisons: use only inequalities (less than, less or equal to, greater than, greater or equal to) instead of strict equality • Guidelines regarding conditional compilation and “pre-processing” • Systematic checking of return conditions (success / failure) Documentation, and, when possible, automation of the production of executable code (makefiles). Avoidance of side effects not apparent from functions signatures. When such side effects exist, guidelines to document them. Bracketing when operators precedence is not absolutely obvious. Catching of supposedly impossible situations (e.g., a “default” case in C “switches”). Use of “wrappers” for critical modules, in particular to check pre- and post-conditions and return conditions. Coding guidelines to comply with known compiler errors and limits set by compiler assessment.
---	---

HDL Coding Standards for Safety-Critical Designs

- RTCA/DO-254 – Airborne electronic hardware
 - Includes guidance for HDL design but lacks detailed coding guidelines
 - Other certification documents require the use of coding standards
- "New" standards for security
 - Language vulnerabilities
 - Security requirements for FPGA implementations
 - ISO/IEC 19790:2025
- NASA guidance document for use of VHDL
- RMM – Reuse Methodology Manual

Using HDL in Avionics

- HDL coding standards required for proper use
- Conformance to those standards should be established
- Includes but are not limited to
 - Comment, style and naming rules,
 - Traceability information for the HDL files (i.e. inclusion of actual file names, document and requirement references if appropriate),
 - Guidelines to ensure the design will synthesize properly,
 - Guidelines to exclude or limit the use of certain types of constructs,

Using HDL in Avionics cont. I

WB

- Rules to address the limits of design and verification tools,
- Guidelines for structure within the HDL (separation between different functions, limits on module sizes),
- Rules to address technological constraints,
 - E.g. to avoid duplicating code for speed

Using HDL in Avionics cont. II

- Guidelines to address specific features (i.e., for synchronous designs, for interfaces with asynchronous signals for management of resets),
- Guidelines to organize the text to improve testability,
- Guidelines to reuse lessons learned from previous developments.

Independence

- For the highest assurance levels, validation and verification activities must be performed independently
- Reviewing the code shall be performed independently for the highest design assurance levels and
 - should be performed for the lower design assurance levels as well

Best Practice for HDL use in Avionics

- Huge effort performed during 2009-2010
- 24 avionics companies contributed, coordinated by Michelle Lange (employed by Mentor Graphics at that time)
- Four classes
 - Coding practices (CP) – 14 rules
 - Safe synthesis (SS) – 21 rules
 - Design reviews (DR) – 13 rules
 - Clock domain crossings (CDC) – 1 rule
- Three different levels of warnings
 - Errors, warnings, and notes

Saab used to have approximately 100 rules and 70 guidelines at that time

Class 1 – Coding Practices

- **CP1 - Avoid Incorrect VHDL Type Usage (Error)**
 - Check for the incorrect use of types, incompatible bounds, and/or constraints
- **CP2 – Avoid duplicate signal assignments (Warning)**

```
same_signal <= '0'; -- Default assignment
IF (reset = '1') THEN
    same_signal <= '1'; -- This is NOT a duplicate assignment.
END IF;
IF (reset = '1') THEN
    -- Default Reset Values
    beep <= '0';
    -- more lines
    beep <= '1'; -- Duplicate assignment maybe a mistake
ELSIF (falling_edge(clk)) THEN
```

Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 1 – Coding Practices cont.

- **CP3 – Avoid Hard-Coded Numeric values (Warning)**

- Typos in hard-coded numeric values will not be detected
- Limits reuse
- Use constants (or generics) instead

```
PACKAGE dcc_pkg is
```

```
-----
```

```
-- constant declarations
```

```
-----
```

```
CONSTANT CPU_ADR_WIDTH : INTEGER := 16; -- cpu address bus width
```

```
CONSTANT CPU_DATA_WIDTH : INTEGER := 32; -- cpu data bus width
```

```
...
```

```
ENTITY dual_clock_cache_struct is
```

```
PORT(
```

```
cpu_addr : IN std_logic_vector( 15 DOWNT0 0 );
```

```
cpu_di : IN std_logic_vector(CPU_DATA_WIDTH-1 DOWNT0 0); --32-bit
```

```
cpu_clk : IN std_logic;
```

Violation

Sources:

Graeme Jessiman, Mentor Graphics, "D0-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 1 – Coding Practices cont.

- **CP4 – Avoid Hard-Coded Vector assignment (Note)**

Example:

WHEN OTHERS =>

clk_div_en <= '0';

xmitdt_en <= '0';

ser_if_select <= "00000000";

ser_in_select <= (OTHERS => '0'); -- This is preferred

Violation



Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 1 – Coding Practices cont.

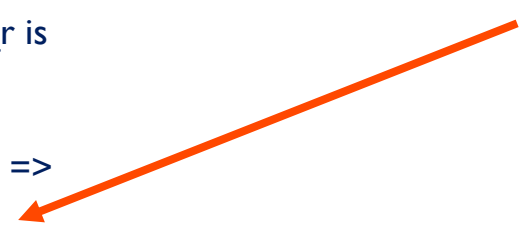
- **CP5 - Ensure Consistent FSM State Encoding Style (Error)**
 - Finite State Machines (FSM) should be consistently used
 - FSM state types should not be hard-coded, unless unavoidable

```

TYPE cpu_sm_state_type IS (
  IDLE, -- reset & default state
  START_OP,
  WRITE_DATA,
  DO_READ,
  WAITMEM,
  STALL_WAIT,
  DO_RD -- Note, FSM states are encoded as enumerated type
);
P_CPU_SM_NEXT_STATE : PROCESS( ...)
BEGIN
  CASE current_state_r is
    WHEN IDLE =>
      ...
    WHEN START_OP =>
      ...
    WHEN "0011" =>
      ...
  END CASE;
END PROCESS P_CPU_SM_NEXT_STATE;

```

Violation, inconsistent state encoding!



Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 &
 Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10
 (modified 9/13/10)

Class 1 – Coding Practices cont.

- **CP6 – Ensure Safe FSM transition (error)**
 - a. An FSM should have a defined reset state.
 - b. All unused (illegal or undefined) states should transition to a defined state, whereupon this error condition can be processed accordingly.
 - c. There should be no unreachable states (i.e., those without any incoming transitions) and dead-end states (i.e., those without any outgoing transitions) in an FSM.

Sources:

Graeme Jessiman, Mentor Graphics, "D0-254 Coding Checks For RTL Code," May 2016 &
Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10
(modified 9/13/10)

Class 1 – Coding Practices cont.

- **Other FSM guidelines**

- FSMs should implement all 2^n states without rest states where possible. This prevents FSM to get stuck in a rest state by accidental bit-flips
- State coding should minimize the number of bit-flips at the state transitions, preferably single bit-flips, which may reduce power and global noise spectra. Simple state machine could use Gray or Johnson coding if no error-detection mechanisms are used in the design
- One-hot-coding should never be used unless error-detection mechanisms are implemented, even though the usage of one-hot encoding can be efficient in some implementations

Class 1 – Coding Practices cont.

- **Other FSM rules**

- Use a distinctive suffix for state variable names, `<_current_state>` for the current state, and `<_next_state>` for the next state
- If using graphic front-end tools when designing FSMs, be sure to keep consistency in the way the tool's parameters are set concerning code generation. For example, if synchronous Mealy is chosen for all FSMs in the design, be sure to explicitly document every deviation from this

Exercise 2 –

Example from Best Practice for HDL use in Avionics – **Can you find all 5 violations?**

- CP14 Avoid unused declarations

14. Avoid Unused Declarations (CP14)

All declared objects should be used.

Check for objects that have been declared, but are never used (i.e., read from or assigned to). Unused declared objects are considered dead code. Dead code can be detrimental when a design is reused. The dead code could be inadvertently activated during the code base port.

Default severity: Warning

Example:

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY unuseddecl IS
    PORT (
        in1 : IN std_logic_vector (3 DOWNT0 0);

        out1,
        out2 : OUT std_logic_vector(3 DOWNT0 0)

    );
END;
ARCHITECTURE unuseddecl_rtl OF unuseddecl IS
    SIGNAL temp1, temp2 : std_logic;

BEGIN

    temp1 <= in1(0);
    out2(3) <= temp2;
END;
```

Source:

Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Exercise 2 –

Example from Best Practice for HDL use in Avionics – **Can you find all 5 violations?**

- CP14 Avoid unused declarations

14. Avoid Unused Declarations (CP14)

All declared objects should be used.

Check for objects that have been declared, but are never used (i.e., read from or assigned to). Unused declared objects are considered dead code. Dead code can be detrimental when a design is reused. The dead code could be inadvertently activated during the code base port.

Default severity: Warning

Example:

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY unuseddecl IS
  PORT (
    in1 : IN std_logic_vector (3 DOWNT0 0);-- Violation
    out1, -- Violation out1 not used in RTL code below,
    out2 : OUT std_logic_vector(3 DOWNT0 0) -- Violation
    -- out2(2 DOWNT0 0) not used in RTL code
  );
END;
ARCHITECTURE unuseddecl_rtl OF unuseddecl IS
  SIGNAL temp1, temp2 : std_logic;

BEGIN

  temp1 <= in1(0); -- Violation Signal 'temp1' is never used
  out2(3) <= temp2; -- Violation Signal 'temp2' is never assigned
END;
```

Source:
Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 2 – Safe Synthesis

- **SS5 – Avoid multiple waveforms**

- Only one waveform should exist on the right-hand side of a signal assignment
- **Example**

```
nRW <= '1' AFTER clk_prd, '0' AFTER 8 * clk_prd;  
-- Violation, Only first waveform element used for synthesis
```

Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 3 – Design Reviews

- **DR6 - Ensure Consistent Indentation (Note)**
- **DR8 - Avoid Large Design Files (Warning)**
- **DR10 - Ensure Consistent File Header (Warning)**
- **DR11 - Ensure Sufficient Comment Density (Warning)**

Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Class 3 – Design Reviews

- Header example:

```

-----
-- VHDL Architecture file/module- name
-- Abstraction: beh, str, rtl
-- Library: compile library
--
-- Company, Division/department
-- Legal: confidentiality, copyright, restrictions on reproduction
--
-- Originally created: date
-- Tools: tool(s)
-- Author: name, e-mail: mail
--
-- Implemented requirements: Set of requirements
--
-- Description: description of function, limitations, errors and
-- list of key-features
-----
-- Revision History:
--
-- Version Author Date Changes
-- X.X Handle YYMMDD Explanation
-----

```

Note! Not all changes should be added. Version control by other means is used for that purpose. The changes in the header should only be those made after formal reviews of the code or when the code is updated after being used and a problem has been identified and corrected!

Class 4 - CDC

- **CDC to be analyzed if applicable**
- 2008 FAA SW and AEH presentation “Mitigating the Dangers of Multi-Clock Designs”
- “Automating Clock-Domain Crossing Verification for DO-254 (and other Safety-Critical) Designs” a whitepaper developed by Mentor Graphics

Sources:

Graeme Jessiman, Mentor Graphics, "DO-254 Coding Checks For RTL Code," May 2016 & Michelle Lange et al, "Best Practice VHDL Coding Standards for DO-254 Programs," DO-254 Users Group, Position Paper, DO254-UG-001, Revision 1a, completed 2/26/10 (modified 9/13/10)

Thank You

- Don't forget to ask the company you will work for if they are using programming subsets and if they use coding standards
 - If it is a company developing any reliable product and they don't have coding standards for both software and hardware, make sure they will